src/vite-env.d.ts

Kind: modified · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T21:21:59.533Z

Overview

`src/vite-env.d.ts` now expands the Electron API surface and global window typings.

- New imports for Git-related types are added (lines 21-31).
- The `ElectronAPI` interface receives dozens of additional methods covering knowledge-graph handling, Git tooling, GitHub integration, workspace management, and progress callbacks (changed ranges 207-374).
- A global `Window` interface is declared to expose `electronAPI` (lines 378-380).
- An `export {};` guard is appended to enforce module scope (line 383).

Key changes

- *Imports
- – `import type { GitRepoStatus, GitToolingStatus, GithubPullRequestSummary, GithubRepoSummary, LocalRepoMatch, PrCompareRoots, PublishExecutePayload, PublishExecuteResult, PublishPreview } from "./app/gitTypes";` (lines 21-31).
- *API surface
- – `ElectronAPI` now includes methods such as `buildKnowledgeGraph`, `readKnowledgeGraph`, `gitDetectTooling`, `gitRepoStatus`, `githubListRepos`, `workspaceCreate`, `workspaceSetActive`, `gitBlameAtRef`, and many others (see changed ranges 207-374).
- *Progress callbacks
- – `onKnowledgeGraphProgress` and `onEngineProgress` are added to expose runtime progress.
- *Global window
- – `declare global { interface Window { electronAPI?: ElectronAPI; } }` (lines 378-380).
- *Export guard
- – `export {};` (line 383).

Impact

- *Type safety
- – TypeScript consumers now see a richer API; missing imports may cause compile errors if not updated.

- *Runtime expectations
- – Renderer calls to the new methods will throw `undefined` if the corresponding main-process implementations are absent.
- *Maintainability
- – The interface now contains ~30 new members, increasing the surface that requires documentation, testing, and future refactoring.
- *Compatibility
- – Existing code that imports `ElectronAPI` will now see additional members; older Electron builds may not support all new methods, potentially breaking backward compatibility.
- *Observability
- – New progress callbacks provide hooks for UI feedback but require careful cleanup to avoid memory leaks.

Risks & follow-ups

- *Unimplemented methods
- – Verify that every new `ElectronAPI` member has a corresponding implementation in the main process; otherwise, renderer calls will fail.
- *Type mismatches
- – Ensure the imported Git types are correctly exported from `./app/gitTypes`; missing re-exports will break compilation.
- *Test coverage
- – Add unit tests for the new API surface, especially for `buildKnowledgeGraph` and GitHub integration paths.
- *Performance
- – Monitor the overhead of the new progress callbacks; excessive event emission could degrade UI responsiveness.